

Plan de Proyecto: Agente IA de “Protección de Imagen Personal”

Este proyecto propone un agente de IA capaz de analizar imágenes proporcionadas por usuarios para detectar **información personal sensible** (por ejemplo, números de DNI, direcciones, placas/patentes de vehículos) y alertar al usuario antes de que dicha información se publique. A diferencia de los filtros tradicionales que buscan contenido explícito, aquí el enfoque está en **proteger la privacidad** al encontrar datos personales visibles en las imágenes del usuario.

1. Arquitectura Lógica

Componentes Principales: La solución sigue una arquitectura cliente-servidor completamente en Azure con estos componentes:

- **Frontend Web (SPA):** Aplicación web ligera (Single Page Application) que permite al usuario cargar imágenes y visualizar los resultados de la detección. Puede ser una página HTML/JavaScript sencilla o una SPA con un framework (React, Angular, Vue) minimalista.
- **API REST Backend (Spring Boot):** Servicio backend en Java 17 con Spring Boot, desplegado en Azure App Service. Expone endpoints REST para que el frontend envíe imágenes. Orquesta el flujo de análisis llamando a servicios externos (OCR) y ejecutando la lógica de detección.
- **Azure Computer Vision (OCR):** Servicio de Azure Cognitive Services que extrae texto de las imágenes. La API de OCR de Azure es capaz de leer texto visible en imágenes (por ejemplo, texto en documentos o letreros en una foto) y convertirlo a texto plano ([Quickstart: Optical character recognition \(OCR\) - Azure AI services | Microsoft Learn](#)). Usaremos su modo síncrono optimizado para imágenes generales (versión OCR 4.0) para obtener rápidamente el texto de la imagen subida ([Quickstart: Optical character recognition \(OCR\) - Azure AI services | Microsoft Learn](#)).
- **Módulo de Detección de Patrones:** Lógica custom en el backend (parte del servicio Spring Boot) que recibe el texto extraído por OCR y busca patrones indicativos de datos sensibles. Utilizará expresiones regulares u otras reglas para identificar formatos de DNI, números de placa vehicular, direcciones (p. ej. presencia de palabras como "Calle", "Av.", números de casa, etc.).
- **Base de Datos (Azure SQL):** Base relacional en Azure SQL Database para almacenar los resultados de los análisis. Guarda registros de cada imagen analizada: texto extraído (opcionalmente), qué datos sensibles fueron detectados y cuándo, u otra metadata. Esto permite auditoría o consulta posterior si se requiere.

Flujo de Procesamiento: A continuación se describe el paso a paso desde que el usuario carga una imagen hasta que obtiene el resultado:

1. **Carga de la imagen:** El usuario selecciona una imagen en el **frontend web** y hace clic en "Analizar/Proteger". La SPA envía la imagen (por ejemplo, mediante una petición HTTP POST multipart/form-data) al endpoint REST del backend.
2. **Recepción en API y llamada a OCR:** El **API REST (Spring Boot)** recibe la imagen en una petición POST. El backend inmediatamente invoca al servicio **Azure Computer Vision OCR** (vía una llamada HTTP REST o SDK) enviándole la imagen. Dado que el procesamiento de imagen será *síncrono*, la API espera la respuesta de Azure en la misma petición.
3. **Extracción de texto:** Azure Computer Vision procesa la imagen y extrae cualquier texto encontrado (por ejemplo, números, letras como podrían ser los de un DNI o matrícula). Retorna ese texto detectado al backend ([Quickstart: Optical character recognition \(OCR\) - Azure AI services | Microsoft Learn](#)) en formato JSON (p. ej., una lista de strings o líneas de texto).
4. **Detección de información sensible:** El backend toma el texto extraído y ejecuta el **módulo de detección de patrones**. Este módulo aplica algoritmos sencillos (expresiones regulares/patrones predefinidos) para buscar datos sensibles:
 - **DNI:** Por ejemplo, detecta secuencias numéricas de 8 dígitos (formato típico de DNI en varios países) o formatos específicos (8 dígitos + letra, si aplicara a DNI de España).
 - **Placas de auto:** Busca patrones alfanuméricos que coincidan con matrículas vehiculares (por ej., combinaciones de 3 letras + 3-4 números, u otros formatos según país).
 - **Direcciones:** Identifica palabras clave como "Calle", "Av.", "Street", "Jr.", etc., junto a números o nombres de lugar, que sugieran una dirección postal.

El resultado de esta fase es un listado de qué tipo de datos sensibles fueron encontrados (y opcionalmente, los valores concretos encontrados, p. ej. "DNI detectado: 87654321").
5. **Almacenamiento de resultados:** El backend guarda un registro en la **base de datos Azure SQL** con la información de la imagen analizada: qué tipos de datos se detectaron, marca de tiempo y quizás un identificador o hash de la imagen (o del usuario, si hubiera gestión de usuarios). Esto permite

mantener un historial de análisis. (Nota: en un entorno real, habría consideraciones de privacidad sobre almacenar el texto extraído; para el hackathon se puede almacenar con fines de demostración).

6. **Respuesta al frontend:** Finalmente, el API REST devuelve una respuesta al **frontend** con los resultados del análisis (por ejemplo, un JSON indicando {"contiene_dni": true, "contiene_placa": false, "contiene_direccion": true} y detalles opcionales). El frontend recibe esta respuesta en la misma llamada que envió (proceso síncrono).
7. **Visualización al usuario:** La SPA muestra al usuario los resultados de forma comprensible. Por ejemplo: “**Datos sensibles detectados:** Se encontró un DNI y una dirección en la imagen”, destacando quizás cuáles. El usuario podría entonces decidir editar o no publicar esa imagen.

Con este flujo, la arquitectura garantiza que el procesamiento ocurra de manera **rápida y en línea** durante la carga de la imagen (gracias al OCR optimizado de Azure) y mantiene un registro de las detecciones para futuras referencias.

2. Tecnologías Específicas (Stack y Versiones)

En este proyecto se utilizarán las siguientes tecnologías y servicios (todas versiones actualizadas a 2025):

- **Java 17 (LTS):** Lenguaje de programación para el backend. Se elige Java 17 por ser soportado por Spring Boot 3 y ofrecer características modernas (Records, Pattern Matching, etc.) manteniendo estabilidad de LTS.
- **Spring Boot 3.x:** Framework para construir el API REST de forma rápida. Spring Boot 3 (con Spring Framework 6) soporta Java 17 de forma nativa y ofrece starters para web (Spring Web), seguridad, data, etc. Versiones ejemplo: Spring Boot 3.1 o 3.2.
- **Azure Cognitive Services – Computer Vision (OCR API):** Servicio cloud para OCR. Usaremos la API “Read OCR” de **Azure Computer Vision**, versión 4.0 (GA) para lectura síncrona de texto en imágenes ([Quickstart: Optical character recognition \(OCR\) - Azure AI services | Microsoft Learn](#)). Se puede invocar mediante el *SDK de Azure AI* para Java o mediante peticiones REST (usando por ejemplo Spring WebClient o RestTemplate). Requiere una **clave de API** y URL de endpoint proporcionados por Azure.
- **Algoritmos de Detección (Regex/Lógica):** En el backend, utilizaremos **expresiones regulares** y lógica Java simple para identificar patrones. Por ejemplo, usar la clase Pattern de Java para compilar regex de DNI (\b\d{8}\b para 8 dígitos), de placas vehiculares (según formato país, e.g. \b[A-Z]{3}\d{3}\b para formato AAA111) y palabras clave para direcciones.

Esta lógica estará encapsulada en un servicio dentro de la aplicación Spring Boot.

- **Azure SQL Database:** Base de datos relacional en la nube de Azure, basada en SQL Server. Se elige por ser un servicio PaaS administrado, compatible con herramientas SQL conocidas ([Azure SQL - Family of SQL Cloud Databases | Microsoft Azure](#)). No requiere manejar la infraestructura y permite escalar fácilmente. Usaremos probablemente la edición *Free/Basic* para desarrollo. Conectividad vía JDBC (driver Microsoft SQL Server para Java).
- **Azure App Service (Java):** Plataforma de despliegue en Azure para aplicaciones web. Alojará la aplicación Spring Boot (se puede desplegar el *JAR* o usar contenedor Docker). Azure App Service simplifica el despliegue continuo y ofrece escalabilidad automática, TLS, etc., sin administrar servidores.
- **Frontend Web (SPA):** Para la interfaz de usuario, se propone una SPA muy simple. Tecnologías posibles:
 - *Opción 1: React 18* (o superior) con JavaScript/TypeScript, construyendo un form de subida de archivo y resultados. Create-React-App o Vite podrían inicializar rápidamente el proyecto.
 - *Opción 2: Vue 3 o Angular 15+*, si el equipo está más cómodo con esos frameworks.
 - *Opción 3: HTML5 + JavaScript vanilla:* incluso un enfoque sin framework (usando un simple `<input type="file">` y código JS para llamar a la API via fetch) sería suficiente dado lo limitado del front. Cualquier opción deberá soportar una llamada multipart para enviar la imagen y mostrar el JSON de respuesta. Se puede usar **Bootstrap 5** u otro CSS framework ligero para darle estilo rápido.
- **Herramientas de Build/DevOps:**
 - **Maven 3.8+:** Gestión de dependencias y build del proyecto Java. Se proveerá un pom.xml con los starters de Spring (web, azure, data, etc.).
 - **Docker (Opcional):** Para contenedorar la app backend y/o correr una instancia local de SQL Server si se desea. Por ejemplo, usar Docker para levantar Azure SQL Edge/Developer localmente durante desarrollo, o para ejecutar el backend en un contenedor consistente.

- **IDE y Otros:** IntelliJ IDEA / VS Code para desarrollo de Java; VS Code/CLI para frontend. Postman o cURL para pruebas rápidas de la API. Git para control de versiones.

(Todas las versiones se elegirán lo más recientes posibles a la fecha, asegurando compatibilidad: p. ej., Spring Boot 3 requiere Java 17+, Azure Cognitive Services SDK en su última versión.)

3. Plan de Desarrollo (Fases para Construir el MVP)

Dado que se trata de un hackathon (desarrollo contrarreloj), conviene dividir el trabajo en fases cortas tipo sprints, priorizando primero una versión **MVP** funcional. A continuación un posible plan:

- **Fase 1: Configuración del Proyecto y OCR Básico**
 - Inicializar el proyecto **Spring Boot** (usar Spring Initializr para generar la estructura con dependencias de Web, posiblemente Spring Data JPA, etc.).
 - Configurar en Azure el **servicio de Computer Vision**: crear el recurso Cognitive Service, obtener la *API Key* y *endpoint URL*.
 - En el backend, añadir la configuración para conectarse al OCR (clave en properties o variable de entorno). Implementar un controlador REST básico con un endpoint de prueba (por ejemplo, /api/testOCR) que envíe una imagen de prueba fija al servicio Azure OCR y logre imprimir/retornar el texto detectado. **Objetivo:** verificar que la comunicación con Azure OCR funciona.
 - Probar este flujo con una imagen conocida (puede ser un DNI o texto simple) usando Postman o un test unitario. Si funciona, ya tenemos la columna vertebral lista (subida de imagen -> OCR -> respuesta texto).
- **Fase 2: Implementación de Detección de Patrones**
 - Desarrollar el módulo de **detección de datos sensibles** en el backend. Crear una clase de servicio (e.g., SensitiveDataDetector) con métodos que reciban el texto OCR y apliquen regex para DNI, placas y búsqueda de palabras clave para direcciones.
 - Definir las **expresiones regulares** necesarias: por ejemplo, patrón DNI de 8 dígitos (\d{8}), patrón de DNI con letra si aplica, patrones de matrículas (ej. \b[0-9]{4}[A-Z]{3}\b para placas españolas modernas, o variantes), patrones de dirección (buscar palabras como "Calle",

"Av", "Street", etc., seguido de número). Mantener los regex simples pero eficaces para demo.

- Integrar este detector en el flujo del controlador principal: después de obtener el texto del OCR, pasar ese texto al detector y capturar los resultados (ej., booleans de si encontró DNI, etc., y posiblemente las cadenas encontradas).
- Formatear la **respuesta JSON** del endpoint principal para incluir estos resultados de detección. Por ejemplo:
- Entre los tipos de datos sensibles que se detectan, tenemos por ejemplo: dni, placas, direcciones, etc. (Según la arquitectura lógica)
- {
- "tipo_dato_detectado": "dni",
- "valor_dato_detectado": "87654321"
- }

(Se pueden ajustar los campos según lo que se quiera mostrar; el MVP podría incluso solo dar flags true/false por tipo).

- Probar localmente con la imagen de prueba y ajustar regex hasta tener detecciones correctas. En esta fase, el *frontend* aún puede ser Postman enviando imágenes.

- **Fase 3: Integración de Base de Datos**

- Levantar una instancia de **Azure SQL Database** (o usar una existente) y obtener cadena de conexión (JDBC URL, usuario y contraseña). Alternativamente, para acelerar, primero usar una base de datos en memoria **H2** durante el desarrollo local y luego apuntar a Azure SQL para la demo final.
- Configurar la conexión en Spring Boot (en application.properties añadir URL JDBC, usuario, pass, dialecto SQLServer). Incluir dependencia de **driver JDBC de SQL Server** (por ejemplo com.microsoft.sqlserver:mssql-jdbc).
- Crear una entidad JPA (ej. ImagenAnálisis) con los campos correspondientes (id, fecha, flags de detección, texto extraído si se guarda). Crear también un **repositorio Spring Data JPA** para esa entidad, para facilitar operaciones de guardado.

- Modificar el flujo del controlador de análisis para, tras obtener los resultados de detección, **persistir** un registro en la base. Guardar los campos relevantes. (En un hackathon, manejar errores mínimos; asumir que la inserción funciona).
- Verificar guardado ya sea mediante logs o consultando la base (p. ej., con Azure Data Studio o el portal de Azure) para asegurarse de que los datos se están almacenando correctamente.

- **Fase 4: Desarrollo del Frontend Mínimo**

- Levantar un proyecto base para el **frontend**. Si se opta por React: usar create-react-app o Vite; si es otra tecnología, crear la estructura mínima. Alternativamente, simplemente crear una página index.html manual con JavaScript.
- Implementar en el frontend una interfaz con un botón o drag-and-drop para seleccionar la imagen y enviarla. Utilizar la **API Fetch** o una librería Axios para hacer la llamada POST al endpoint /api/scan. Asegurarse de enviar la imagen en el formato correcto (multipart form).
- Mostrar el **resultado** en la misma página sin refrescar (característica de SPA): por ejemplo, después de la petición, procesar el JSON devuelto y renderizar un listado de las detecciones. P.ej., pintar un mensaje rojo si se detectó DNI: " ⚠ Se detectó un DNI en la imagen: 87654321".
- Darle un mínimo de estilo para la presentación (CSS básico o Bootstrap). No invertir demasiado tiempo en estética, pero asegurarse de la claridad del resultado (uso de íconos o color para alertar, etc.).
- Realizar pruebas integrales: correr el backend local (o en Azure) y el frontend local, subir imágenes reales de prueba desde la interfaz y confirmar que todo el flujo funciona (la imagen viaja, el backend responde, el front muestra la info).

- **Fase 5: Pruebas Finales y Despliegue en Azure**

- Hacer pruebas con varias imágenes de distintos tipos para verificar la robustez del MVP. Por ejemplo: una foto de un DNI borroso (debería detectarlo si es legible), una foto de un auto con la placa visible, una captura de pantalla con una dirección escrita, y también imágenes sin datos sensibles para ver que sale todo en *false*.

- Ajustar rápidamente cualquier punto débil (por ejemplo, refinar alguna expresión regular si hay muchos falsos positivos/negativos en pruebas).
- Preparar el entorno Azure para la demo: desplegar la aplicación Spring Boot en **Azure App Service**. Esto puede hacerse empaquetando con Maven (mvn package) y subiendo el JAR, o usando un contenedor Docker y Azure Container Registry seguido de App Service. Asegurarse de configurar las **variables de entorno** en Azure App Service: URL y key del OCR, cadena de conexión de la base de datos, etc.
- Desplegar también el **frontend**. Si es una app React/Angular, construir (npm run build) y servir los archivos estáticos. Opciones: hospedarlos en Azure Storage Static Websites o en **Azure Static Web Apps**, o más sencillo, habilitar CORS en el backend y servir el frontend localmente durante la demo. Para simplicidad en hackathon, incluso se podría incorporar el frontend estático en el mismo App Service (por ejemplo, colocando los archivos estáticos en src/main/resources/static de Spring Boot para que se sirvan desde el mismo servidor).
- Probar la versión desplegada end-to-end: acceder a la URL pública del frontend (o ejecutar frontend local contra la API en Azure) y verificar que todo funciona en la nube (especialmente latencia de OCR, conexión a DB remota, etc.).

Cada fase puede solaparse en parte si el equipo tiene varios integrantes (por ejemplo, alguien puede empezar el frontend en paralelo a que otro integra la base de datos). El objetivo es que al final de la hackathon se tenga un MVP funcional desplegado o al menos demostrable localmente, cubriendo el caso principal: **detectar datos sensibles en imágenes y alertar al usuario**.

4. Endpoints Principales del API REST

A continuación se definen los endpoints REST que expone el backend (Spring Boot). Todos prefijados por, por ejemplo, /api. Los más relevantes para el MVP son:

- **POST /api/scan – Analizar imagen.**

Descripción: Endpoint principal que recibe una imagen para escanear. El cliente (frontend) envía la imagen como parte de la petición (usando multipart/form-data con un campo file, o bien en el cuerpo binario directamente). El servidor procesa la imagen (OCR + detección) de forma **síncrona** y responde con un JSON con los resultados.

Cuerpo: archivo de imagen (JPEG, PNG, etc.).

Respuesta: JSON indicando qué datos sensibles fueron encontrados. Por ejemplo:

- {
- "contiene_dni": true,
- "contiene_placa": false,
- "contiene_direccion": true,
- "detalles": {
- "dni": "87654321",
- "direccion": "Av. Siempre Viva 742"
- }
- }

(La estructura exacta puede ajustarse; podría ser simplemente una lista de tipos detectados). Si no se detecta nada, podría devolver `contiene_*` todos en `false` y/o un mensaje "No se hallaron datos sensibles".

- **GET /api/scans – Listar análisis previos** (opcional para demo).
Descripción: Retorna un listado de los últimos análisis realizados, obtenidos de la base de datos. Esto serviría para mostrar un historial de imágenes escaneadas. Cada entrada podría incluir id, fecha y qué se detectó. En un entorno real, probablemente se filtraría por usuario autenticado; pero en el MVP (sin autenticación), este endpoint se usaría solo para propósitos de demostración o verificación.
Respuesta: JSON array, e.g.:

- [
- { "id": 5, "fecha": "2025-04-26T21:00:00Z", "contiene_dni": true, "contiene_placa": false, "contiene_direccion": true },
- { "id": 6, "fecha": "2025-04-26T21:05:00Z", "contiene_dni": false, "contiene_placa": true, "contiene_direccion": false }
-]

(Si el tiempo es corto, este endpoint no es imprescindible. El foco es el POST; pero tenerlo podría sumar valor mostrando el uso de la DB).

- **GET /api/scans/{id} – Obtener resultado por ID** (opcional).
Descripción: Retorna el detalle de un análisis específico (incluyendo posiblemente los textos extraídos y datos detectados) buscando por

identificador único. Útil si quisiéramos que el frontend solicitara los resultados en una segunda llamada en vez de recibirlos inmediatamente, o para una pantalla detallada. En el MVP, dado que el procesamiento es síncrono y se devuelve todo en la respuesta del POST, este endpoint no es estrictamente necesario, pero podría existir para completar el diseño RESTful.

- **Otros endpoints:** Se puede exponer un **GET /api/health** o similar para verificar rápidamente que el servicio esté arriba (útil tras desplegar en Azure, para probar que la app responde). Esto podría devolver simplemente `{ "status": "ok" }`.

Todos los endpoints responderán con código HTTP 200 en caso de éxito en el análisis. En caso de error (por ej., si Azure OCR falla o hay un problema con la imagen), se pueden manejar errores retornando códigos 4xx/5xx apropiados con un mensaje. Sin embargo, para el hackathon, el manejo de errores puede ser rudimentario, enfocándonos en el caso exitoso.

5. Esquema de la Base de Datos

La base de datos será **relacional** (Azure SQL). Para el MVP, un esquema simple es suficiente, centrado en una tabla principal que registre los análisis de imágenes. A continuación una propuesta de tabla:

Tabla AnalisisImagen (registra cada imagen analizada y resultados):

Campo	Tipo	Descripción
id (PK)	int AUTO_INCREMENT	Identificador único del análisis.
fecha_hora	datetime	Fecha y hora en que se realizó el análisis.
texto_extraido	TEXT	Texto bruto extraído de la imagen por OCR (opcionalmente almacenado para referencia o debug).
contiene_dni	boolean	TRUE si se detectó un número de DNI en la imagen.
dni_detectado	varchar(20)	Valor del DNI encontrado (ej. "87654321" o "87654321-X"), null si ninguno.

Campo	Tipo	Descripción
contiene_placa	boolean	TRUE si se detectó una placa/patente de auto.
placa_detectada	varchar(15)	Texto de la placa vehicular detectada (ej. "ABC123"), null si ninguna.
contiene_direccion	boolean	TRUE si se identificó alguna dirección postal en el texto.
direccion_detectada	varchar(255)	Dirección identificada (ej. "Av. Siempre Viva 742"), null si ninguna.

Algunos puntos sobre este esquema:

- Los campos *_detectado guardan la primera ocurrencia encontrada (o podrían guardar múltiples separadas por coma si hubiera varias en una imagen). Para simplificar, MVP puede asumir una sola ocurrencia relevante por tipo.
- Almacenar texto_extraido completo puede ser útil para depuración o mejorar algoritmos, pero si se tienen preocupaciones de privacidad, se podría omitir en producción. En un hackathon, incluirlo está bien para inspeccionar resultados.
- Podría haber una tabla adicional Imagen para almacenar metadata de la imagen (nombre de archivo, usuario, etc.) y otra tabla Detecciones relacional (una fila por cada dato detectado, vinculada al análisis). Sin embargo, dado el tiempo, **una sola tabla** consolidada como arriba es más rápida de implementar y suficiente para la demostración.
- Si hubiera gestión de usuarios, agregar un campo usuario_id para asociar cada análisis a quién lo subió. En este MVP no se contempla autenticación, así que no se incluye.

El acceso a esta base se hará mediante **Spring Data JPA** en el backend, lo que permitirá insertar registros simplemente llamando a métodos del repositorio (por ejemplo analisisRepo.save(analisis)), y consultar si se implementa el endpoint de historial.

6. Instrucciones para Levantar el Entorno Local

Para poder desarrollar y probar el proyecto localmente antes de desplegar en Azure, se deben seguir estos pasos iniciales:

1. **Prerequisitos de Instalación:** Asegurarse de tener instalado en el equipo de desarrollo:
 - **JDK 17** (Java Development Kit 17).
 - **Maven 3** (para compilar y gestionar el proyecto).
 - (Opcional) **Node.js 18+** si se va a construir un frontend con Node (React/Angular).
 - (Opcional) **Docker** si se planea usar contenedores o una base de datos local en contenedor.
2. **Clonar el Repositorio:** Obtener el código fuente del proyecto (repositorio Git proporcionado o crear uno nuevo con spring init). Navegar al directorio del backend.
3. **Configurar Variables de Entorno/Propiedades:** El proyecto debe estar configurado para leer ciertos datos sensibles desde el entorno o un archivo de propiedades. Antes de ejecutar, configurar:
 - **Azure Computer Vision Key y Endpoint:** En el archivo `application.properties` (o `.yml`) del Spring Boot, setear las propiedades para la clave y URL del servicio OCR. Por ejemplo:
 - `azure.cognitiveservices.vision.key = YOUR_CV_KEY`
 - `azure.cognitiveservices.vision.endpoint = YOUR_CV_ENDPOINT`

Alternativamente, exportar variables de entorno (p. ej. `AZURE_CV_KEY` y `AZURE_CV_ENDPOINT`) y mapearlas en Spring Boot.

- **Cadena de Conexión de Base de Datos:** Si se va a probar con la base Azure SQL, obtener la cadena de conexión JDBC (formato `jdbc:sqlserver://<servername>.database.windows.net:1433;database=<dbname>;...`) junto con las credenciales (usuario/contraseña) y configurar en `application.properties` las propiedades `spring.datasource.url`, `spring.datasource.username`, `spring.datasource.password`. Asegurarse de que la IP local esté autorizada en el firewall de la base de datos Azure SQL (o usar un túnel/VPN si aplica).
 - *Opción más simple:* Para entorno local, usar una base **H2 en memoria** para pruebas rápidas. Spring Boot puede configurarse para usar H2 cuando se active un perfil dev. Esto evita tener que depender de Azure SQL durante el desarrollo inicial. En este caso, configurar

spring.datasource.url=jdbc:h2:mem:testdb y las propiedades necesarias. Los scripts DDL de creación de tablas se pueden ubicar en src/main/resources/schema.sql para auto-creación.

- **Otros:** Verificar que el puerto de la aplicación (por defecto 8080) no esté en uso; configurar propiedad server.port si se necesita cambiar.

4. Construir y Ejecutar el Backend:

- Ejecutar mvn clean install para compilar el proyecto y descargar dependencias. Si todo compila correctamente, lanzar la aplicación:
- Ejecutar con Maven: mvn spring-boot:run (en la raíz del proyecto backend) o ejecutar la clase principal (Application.java) desde el IDE.
- Verificar en la consola que Spring Boot levanta sin errores. Debería mostrar algo como "Started Application in X seconds. Tomcat started on port 8080".

5. Probar el Backend (solo) Localmente:

- Utilizar **Postman** o curl para hacer una solicitud de prueba: por ejemplo, a http://localhost:8080/api/scan enviando una imagen. En Postman, se puede hacer con Body = form-data, key "file" tipo File, seleccionar una imagen de prueba.
- Observar la respuesta JSON y la consola de la app para asegurarse de que el OCR funcionó (posiblemente aparezcan logs del texto extraído) y que la detección de patrones funcionó. Ajustar cualquier configuración si la conexión a Azure falla (por ejemplo, revisando la clave o endpoint).

6. Levantar el Frontend Localmente: (Si se implementó un frontend separado)

- Navegar al directorio del frontend (p. ej., si se usó React, la carpeta creada por create-react-app).
- Instalar dependencias con npm install.
- Configurar la URL del backend: En el código frontend, asegurarse de que las peticiones apunten a la URL correcta del backend. En desarrollo local será http://localhost:8080/api/scan. Si el frontend y backend corren en puertos distintos, habilitar CORS en el backend

para ese dominio/puerto. (En Spring Boot, esto puede hacerse con `@CrossOrigin` en el controlador, o propiedades de configuración).

- Iniciar la aplicación frontend: `npm start` (React) / `ng serve` (Angular) / abrir el `index.html` en un navegador (si es app estática).
- En un navegador, abrir la aplicación y probar la carga de una imagen a través de la interfaz. Debería mostrar los resultados en pantalla.

7. **Docker (opcional local):** Si se desea ejecutar todo con Docker (por ejemplo, para evitar instalaciones locales):

- Se puede crear un **Dockerfile** para el backend (FROM eclipse-temurin:17-jdk-alpine, copiar el JAR y ENTRYPOINT `java -jar app.jar`). Construir la imagen con `docker build -t image-protect-backend .` y correr con `docker run -p 8080:8080 -e AZURE_CV_KEY=... -e ... image-protect-backend`.
- Para la base de datos, Azure SQL no tiene imagen, pero se puede usar **SQL Server Developer** en Docker: `docker run -e 'ACCEPT_EULA=Y' -e 'SA_PASSWORD=YourPassw0rd' -p 1433:1433 -d mcr.microsoft.com/mssql/server:2019-latest`. Luego cambiar la URL JDBC a `jdbc:sqlserver://localhost:1433;... con user=sa, password=YourPassw0rd`. (Esto solo si se quiere emular la DB localmente; H2 es más simple).
- Verificar conexión del Spring Boot container hacia el container de DB (posiblemente mediante `network docker`). Dado el tiempo de hackathon, usar H2 es menos complejo, por lo que este paso con Docker para DB es opcional.

Siguiendo estos pasos, el entorno de desarrollo local debería estar listo para iterar rápidamente. Una vez todo probado, se podrá desplegar en Azure usando mecanismos similares, ajustando configuraciones para producción.

7. Sugerencias para Acelerar el Desarrollo en el Hackathon

Finalmente, se listan algunas **buenas prácticas y atajos** para maximizar productividad dada la naturaleza de un hackathon:

- **Enfocarse en el MVP primero:** Priorizar tener el flujo principal funcionando de punta a punta lo antes posible (aunque sea de forma básica): es decir, subir imagen -> obtener detección -> ver resultado. Evitar agregar funciones extra (como autenticación de usuarios, procesamiento batch, etc.) hasta no asegurar el caso base.

- **Utilizar Plantillas y Quickstarts:** Ahorrar tiempo aprovechando código de ejemplo. Microsoft ofrece quickstarts para Azure Computer Vision OCR (incluso en Java) que se pueden adaptar ([Quickstart: Optical character recognition \(OCR\) - Azure AI services | Microsoft Learn](#)). Usar Spring Initializr para no configurar desde cero el proyecto. Incluso para el frontend, usar generadores (create-react-app, etc.) para obtener una base funcional rápidamente.
- **Paralelizar tareas:** Si el equipo tiene varios miembros, trabajar en paralelo en frontend y backend desde temprano. Definir un contrato de API (formato de request/response JSON) anticipadamente para que el frontend pueda simular la respuesta mientras el backend se implementa. Asimismo, alguien puede encargarse de configurar los recursos Azure (Computer Vision, SQL, App Service) en segundo plano.
- **Pruebas con datos reales frecuentes:** Conseguir o crear **imágenes de prueba** que contengan datos simulados: por ejemplo, generar una imagen que tenga un número con formato DNI, otra con una matrícula de auto, etc. Probar continuamente el OCR con esas imágenes para ajustar los patrones. (Importante: usar datos ficticios para no exponer información real).
- **Simplificar el Frontend:** Dado el tiempo limitado, mantener la interfaz lo más simple posible. La funcionalidad es más importante que la apariencia. Incluso una página estática con un input file y un área de texto para mostrar JSON es suficiente para demostrar el concepto. Si hay tiempo al final, embellecer con CSS.
- **Manejar configuración de forma segura pero sencilla:** En lugar de invertir tiempo en un sistema complejo de configuración, usar un archivo .properties único o variables de entorno sencillas para cosas como claves Azure. Evitar secreto en código (por seguridad), pero en hackathon es aceptable tener un .env que cargue variables localmente.
- **Uso de H2 para desarrollo:** Ya mencionado, esto agiliza no tener que esperar despliegues de base de datos en cada prueba. Se puede volcar a Azure SQL al final.
- **Logging y depuración:** Añadir logs en el backend alrededor de las etapas (por ejemplo, imprimir el texto devuelto por OCR, imprimir cuándo se detecta algo) ayuda a depurar rápido sin necesidad de adjuntar un depurador. En caso de fallo, se verá rápidamente en qué paso ocurrió.
- **Control de errores minimalista:** Implementar *try/catch* alrededor de la llamada a Azure OCR para manejar posibles errores de conexión o imagen

inválida, de forma que el servicio no se caiga. Retornar un mensaje de error sencillo al frontend (`{ "error": "No se pudo procesar la imagen" }`). No dedicar demasiado tiempo a casos extremos, pero tener al menos esa seguridad para que la demo no se rompa por una excepción no controlada.

- **Limitar el tamaño de las imágenes:** Si es posible, en el frontend o backend, restringir el tamaño de archivo aceptado (por ejemplo < 5 MB) para evitar demoras o tiempo excedido en OCR. Azure Computer Vision tiene límites de tamaño/resolución; verificar documentación (en general 4MB para OCR API). Redimensionar o rechazar imágenes muy grandes en el frontend puede ahorrar tiempo en la llamada.
- **Azure deployment:** Si se planea desplegar, hacer uso de herramientas sencillas: por ejemplo, usar **Azure Maven Plugin** para desplegar directamente a App Service, o usar **GitHub Actions** si el repositorio está en GitHub (Azure tiene acciones pre-hechas para Java deployments). No reinventar el CI/CD en un hackathon, apoyarse en asistentes del portal de Azure o instrucciones paso a paso ya existentes.
- **Comunicación constante en el equipo:** Dado el estrés de un hackathon, asegurar que todos entienden el objetivo y el plan. Revisar brevemente tras cada fase lo que funciona y qué sigue. Mantener divisiones de trabajo claras (por ej., “yo me encargo de la API OCR, tú del frontend upload form, otro de la DB”).
- **Preparar la demostración:** Reservar algo de tiempo antes del final para pensar cómo presentar el proyecto. Por ejemplo, preparar un caso de uso donde se sube una imagen con un DNI visible y mostrar cómo el sistema lo detecta y advierte. Tener a mano varias imágenes de ejemplo para la demo en vivo. Practicar la explicación de la arquitectura brevemente (quizás mostrando el diagrama lógico dibujado en una pizarra o slide).